PlacePatch

One place. One change. One team plan.

Turn a photograph of a local place into a clearly labeled improvement concept, a community proposal, and a reviewable request packet.

The full loop: upload a photo → choose a change → draw an edit area → generate a concept → review and publish → support → export a request.

DECISION	THIS SPRINT
Product	One responsive web app for phones and desktops; one backend; one shared repository.
Frontend	React + TypeScript + Vite. One person owns the entire interface.
Backend	Django REST Framework; typed AI adapters; durable background jobs.
Identity and data	Auth0; Tiger Data PostgreSQL if available, otherwise local PostgreSQL.
AI	Ollama analysis/drafting; ComfyUI image editing; Gemini concept review.
Submission	Cox Social Good. Target side challenges: Gemini + Auth0 + Tiger Data.
Deadline	Sunday October 11, 09:00 EDT. Aim to submit by 07:30.

Read this together for 20 minutes, then split up

This version combines the supplied PlacePatch PDF and your handwritten architecture sketch. It resolves the competing stack options and replaces the old 26-hour schedule. Named tools are planned integrations, not claims that they already work.

Companion kit: README, detailed API contract, machine-readable response schemas, sample responses, per-person Cursor prompts, and acceptance checklist. The PDF is sufficient for the team kickoff; the kit is the implementation reference.

BUILD GUIDE / 01

What everyone is building

Everyone should be able to explain the same user story. Separate modules must produce one complete product.

First demo: a commuter uploads a consented bus-stop photo and requests one shade canopy. PlacePatch proposes a bounded change, edits the selected image region, and creates a labeled before/after concept. The commuter reviews the exact wording and publishes. A second account supports once. The team exports a one-page request.

What the app produces

- An immutable sanitized source photo, separate from every generated concept.
- A specific request tied to a user-supplied place, with uncertainty visible.
- A public proposal showing source and illustrative concept side by side.
- An authenticated count of accounts supporting the proposal.
- A packet containing both images, the request, support count as of export, and a recipient field that may remain unconfirmed.

What success means for this hackathon

A new user completes the entire loop using one real photo and one genuine image edit. A second account can support it; repeated support does not increase the count. The packet contains the correct images and wording. A failed AI call has an honest failure screen.

Keep the intervention narrow

Start with shade only. The earlier example combined shade with an accessible waiting area; that adds design and verification complexity. Other changes can be added after the shade example works. Never claim the generated structure meets engineering or accessibility requirements.

Scope cuts

No separate native app, citywide map, auto-email, AR, construction costs, or official approval claims. Comments and advanced charts are optional. Exporting a packet does not mean it has been sent to an organization.

BUILD GUIDE / 02

Freeze one stack and three entries

The handwritten sketch lists alternatives. This is the recommended single choice to prevent four incompatible implementations.

LAYER	SELECTED	WHY
Mobile + web UI	React / TypeScript / Vite	One responsive app covers phone capture and desktop forum.
Backend API	Django + REST Framework	Owns identity checks, database writes, storage and jobs.
Agent functions	Python modules + explicit state machine	B and C fit naturally beside Django; avoid extra orchestration infrastructure initially.
Images	ComfyUI; configured fallback if local fails	Prove one masked edit on actual hardware early.
Database	Tiger Data PostgreSQL; local Postgres fallback	Same SQL model; avoid a late SQLite-to-Postgres migration.
Auth	Auth0 SPA + backend token verification	Publishing/support need trustworthy account identity.
Hosting	Vultr preferred	Only after local end-to-end flow; requires reachable providers and persistent media.

What this replaces

Do not build Streamlit and React together, or Flutter/React Native alongside the web app. Django is the backend, not a competing React UI. This recommendation supersedes the original PDF's Next.js option for this sprint. If substantial working code already exists, agree any migration before replacing it.

Side challenges: hard maximum of three

Default: Gemini reviewer, Auth0 identity, Tiger Data event analytics. Mercedes and Vultr can be used technically without entering their prizes. To enter either, replace one of the three and record the decision. Only claim integrations demonstrated in the running app. [1]

Before coding: write the four names, integration machine, repository owner, source photo, model choices, and image fallback into the shared decision log.

BUILD GUIDE / 03

Four owners, clear boundaries

One person owning the entire frontend is a good division. The other teammates need to understand its inputs and outputs, not its React implementation.

OWNER	FILES / RESPONSIBILITY	FIRST DELIVERABLE
A: Product + all UI Name: _____	frontend/**. Capture, options, mask brush, progress, slider, review, feed, support, packet screen.	Complete screen flow using labeled sample responses.
B: Agents + review Name: _____	backend/agents/**. Scene understanding, options, edit prompt, review, request draft; Gemini reviewer.	Valid scene + review JSON from one actual photo.
C: Image pipeline Name: _____	backend/imaging/** and workflows/**. ComfyUI, masks, render adapter, retries, metadata, fallback.	One genuine recognizable edit with measured duration and exact workflow.
D: Backend + integration Name: _____	backend/core/**, config/**, deployment. API, Auth0 verification, DB, media, jobs, PDF, merge coordination.	Authorized photo upload/proposal retrieval and persistent unique supports.

Shared ownership decisions

• D owns public routes, migrations, dependency changes and integration merges. B/C expose Python functions instead of inventing separate frontend endpoints.

• A builds the mask brush. C defines mask conventions. D validates and stores the uploaded mask.

• A designs the packet layout; D generates the final PDF from authoritative stored data.

• A wires Auth0 login in the UI; D configures the API audience and validates tokens server-side.

• D owns final submission; another teammate independently checks both submission confirmations.

Done means handed off: another teammate runs it using the README, sees the documented output, and can reproduce the error behavior. A notebook-only success is not integration.

BUILD GUIDE / 04

How the parts connect

The frontend calls only your backend. It never needs direct access to Gemini, ComfyUI, Ollama, or database credentials.

PATH	WHAT MOVES ACROSS IT
A → D	Photo + problem; selected option; PNG mask; publish approval; support action.
D → B	Validated image path + problem. Later: selected option and source/concept pair for review.
B → D	Structured scene options, preservation list, review flags, and editable request text.
D → C	Sanitized source path + same-size mask + approved edit prompt.
C → D	Generated local output path + actual provider/model + seed if available + duration.
D → A	Stable JSON, asset IDs/URLs, background-job stages, authoritative counts and packet.

Example: generating the preview

- A submits the option ID, mask asset ID, and expected proposal revision.
- D validates ownership and input, creates a durable job, and immediately returns jobId.
- A shows the actual job stage and polls about every two seconds.
- D's worker asks B for the edit prompt, C for the image, then B for review and wording.
- D saves the result. A fetches the proposal and displays the comparison and warnings.

Frontend work does not have to wait

A uses a fixture-mode API client returning the agreed sample JSON. D implements the live endpoints with the same shapes. Changing one client setting later should connect the UI; it should not require rewriting the screens.

Fixture rule: show "DEVELOPMENT FIXTURES - NOT LIVE AI" at all times. Fixtures do not prove image generation, authentication, community support, or sponsor success. No fixture writes to the live database.

BUILD GUIDE / 05

Screen-by-screen handoff

This is the shared interface walkthrough. A can explain each row in plain language at the kickoff.

SCREEN / USER ACTION	SERVICE OUTPUT	OWNER / FAILURE
Capture: login, photo, place, problem, consent	Create proposal returns proposal object + scene jobId.	D starts B. Retain form on invalid photo or auth failure.
Suggest: inspect scene, select one change	Scene options with IDs, request text, preserve list and uncertainties.	B via D. Show unclear-input questions rather than guessed facts.
Mask: brush selected image region	Private PNG mask assetId and new revision.	A brush; D validates; C consumes. Reject size mismatch.
Preview: click Generate	202 jobId; queued/running/render/review stages; updated concept asset.	D invokes C/B. Retry or keep draft on failure.
Review: edit text and approve	Current title, source, concept and flags; successful publish state.	A collects approval, D enforces it. Stale revision asks user to reload.
Public proposal: read and Support	Authoritative count, viewerHasSupported, alreadySupported.	D. Login for support; duplicate click does not inflate count.
Community: browse proposals	Published-only list with cursor.	D list/A cards. A useful empty state before first publication.
Handoff: download packet	One-page PDF with source, labeled concept, request and export-time count.	D renderer/A layout. Recipient may say "to confirm".

UI conventions all teammates should know

Use the same source and concept aspect ratio; label both. Show real stages rather than fake percentages. Keep forms editable after errors. All public wording is user-approved. The community feed must never expose drafts, raw private photos or account details.

BUILD GUIDE / 06

The minimum API contract

Base path: /api/v1/. Use the exact route and field names in the companion API_CONTRACT.md. No trailing slash redirects on POST requests.

METHOD / PATH	REQUEST	RESPONSE
POST /proposals	Multipart photo, placeLabel, problemText, consent	201: proposal + jobId
GET /proposals/{id}	ID; token if draft	200: Proposal
GET /jobs/{id}	Owner token	200: status, stage, error
POST /proposals/{id}/mask	PNG mask + expectedRevision	201: assetId + revision
POST /proposals/{id}/preview	optionId, maskAssetId, expectedRevision; idempotency key	202: jobId + proposalId
PATCH /proposals/{id}	title, changeRequest, expectedRevision	200: updated Proposal
POST /proposals/{id}/publish	expectedRevision, approveText, approveConcept, acknowledgedFlagIds	200: published Proposal
POST /proposals/{id}/support	Verified user token; empty body	200: supportCount, viewerHasSupported, alreadySupported
GET /proposals/{id}/packet	Published proposal ID	PDF download

Rules behind every route

D derives identity from a verified Auth0 access token. Never trust a submitted authorId or supporterId. Private proposals/jobs/assets are owner-only. Public reads expose only published content. The database decides status, revisions and counts. A must handle authentication and revision errors explicitly.

State and error conventions

Proposal: draft → ready → published. **Job:** queued → running → succeeded/failed. Exporting is a separate action, not a publication state. All JSON timestamps use UTC; the schedule in this PDF uses EDT.

Error envelope: {"error":{"code":"REVISION_CONFLICT","message":"Reload the proposal.","retryable":false,"fieldErrors":{}}}
HTTP 401 means sign in; 409 means stale content or invalid transition; 503 means unavailable provider.

BUILD GUIDE / 07

Agree on the data, not just screens

Use these objects to discuss the implementation. The companion schemas and fixture JSON files provide full machine-readable examples.

Proposal: the shared unit of work

Identity/state: id, revision, status, createdAt, publishedAt.
User content: title, placeLabel, problemText, changeRequest.
Assets: source, concept (nullable), maskAssetId (nullable).
AI: scene, review, generation, aiDisclosure.
Support: supportCount, viewerHasSupported.

Example responses

```
Preview accepted (HTTP 202):
{"jobId":"job-17","proposalId":"proposal-1"}

Job still processing:
{"id":"job-17","proposalId":"proposal-1",
 "status":"running","stage":"render",
 "createdAt":"2026-10-10T17:00:00Z",
 "updatedAt":"2026-10-10T17:00:01Z","error":null}

Repeated support (HTTP 200):
{"proposalId":"proposal-1","supportCount":2,
 "viewerHasSupported":true,"alreadySupported":true}
```

Prevent integration surprises

• Source and concept are distinct Asset objects with IDs, dimensions, hashes and protected URLs. A missing concept stays null; it is not a fake image.

• Use one casing convention on the wire: camelCase. Python internals may use snake_case.

• Every draft mutation increments revision. Preview and publish include expectedRevision. Stale jobs cannot overwrite newer content.

• Support has a database unique key on proposal + authenticated account. Only actual inserts create support events.

• A contract change requires schema, example, documentation and affected code to change together, with both producer and consumer informed.

BUILD GUIDE / 08

AI and rendering: prove it early

The hardest dependency is producing a recognizable genuine edit. C must test it while A, B and D build their independent pieces.

FUNCTION / OWNER	INPUT → OUTPUT
analyze_scene / B	Sanitized photo + problem → observations, uncertainties, one or two options.
build_edit_prompt / B	Scene + selected option → one edit goal with preservation constraints.
render_edit / C	Source path + mask path + prompt → outputPath, provider, model, seed/null, durationMs, workflowVersion.
review_concept / B	Source + concept + requested change → clear/needs_review/blocked, flag IDs, provider/model.
draft_request / B	Approved option + scene + review → editable title and changeRequest.

Mask agreement

A exports a PNG at the sanitized source's actual pixel dimensions. White means edit; black means preserve. D validates dimensions, ownership and nonempty scope. C adapts it to the ComfyUI workflow's actual mask conventions and verifies alignment. Do not use CSS display dimensions for the mask file.

First test and fallback

By 14:30 EDT, C should have one real edit, its elapsed time, model/workflow metadata and a repeatable command. If the available hardware cannot do it, switch to a configured hosted editor with the same function contract. Do not spend the event installing increasingly large checkpoints.

Where models run

Choose one capable integration machine first. A remote backend cannot reach C's laptop by calling localhost. Before deployment, D/C must choose server-hosted models, a configured cloud provider, or a secured private worker connection. Never expose an unauthenticated image service publicly.

Gemini's specific job: compare the source, concept and request and return structured review flags. It flags uncertainty; it does not certify safety, approve publication, or invent support counts.

BUILD GUIDE / 09

Backend, trust and reliable jobs

Authentication is part of the first live integration, not a late decoration. Real publishing and unique supports depend on it.

Auth0 and authorization

A uses Auth0 React login and requests an access token for your API audience. D validates token signature, issuer, audience and expiry; account identity comes from sub. A UI login badge alone does not protect backend routes. Never use a user ID supplied by the browser as authorization. [2]

Storage and database

Prefer Tiger Data PostgreSQL if D can connect promptly; use local PostgreSQL if blocked. Keep ordinary relational tables for proposals, assets, supports and jobs. Add event analytics only after actual event writes work. Support uniqueness belongs in a real database constraint, including concurrent-click behavior.

Long image jobs

POST preview creates a database job and returns immediately. A separate Django worker claims jobs, calls B/C, records output/error and updates the proposal. D defines timeout/lease and one bounded retry from actual render timing. Restarting the HTTP server must not silently lose all job state. No untracked long-running threads in a request handler.

Privacy and human approval

• Normalize source orientation, strip metadata, and hash a sanitized immutable source. Keep generated concepts separate. Raw uploads are not public assets.

• Use a clean consented demo image. If people/plates/private details are visible, require crop/redaction before analysis or hold the photo. Do not promise automatic detection unless implemented.

• Show review warnings before publication. A blocked review cannot be bypassed by a simple acknowledgment checkbox.

• All concepts and exported packets carry an AI-illustration label. Packet recipient and ownership may remain uncertain.

• No automated emailing or official-status claims. Downloading a packet is not evidence that an office received it.

BUILD GUIDE / 10

One repository, four branches

GitHub does not need to be connected to this chat. D can create the repository manually and invite the other three teammates.

Repository structure

```
README.md                 everyone reads first
AGENTS.md                 rules for coding assistants
frontend/                 A: all React screens and API client
backend/config/           D: Django configuration
backend/core/             D: models, API, auth, jobs, exports
backend/agents/           B: analysis, planning, review
backend/imaging/          C: renderer adapter
workflows/                C: reproducible image workflow
shared/schemas.json       agreed wire response shapes
shared/fixtures/          sample responses for A
config/*.env.example      empty configuration templates
docs/                     contracts, plan, tests, prompts
```

Create once; clone four times

D creates one empty repo named placepatch, copies the companion kit into it, and pushes the initial plan. A scaffolds the frontend once; D scaffolds the backend once. Everyone else pulls those scaffolds instead of making their own app. If a repository already exists, add the kit through a normal branch/PR.

PERSON	BRANCH	MUST NOT EDIT INDEPENDENTLY
A	feat/frontend	Backend routes or migrations
B	feat/agents	Public API or shared dependency files
C	feat/image-pipeline	Frontend, authentication or database schema
D	feat/backend	Other owners' implementations without coordination

Merge and communication routine

Small commits and PRs every 60-90 minutes. D reviews integration order; never wait until midnight to merge everything. Shared contract changes get producer/consumer acknowledgment. Each PR says: what changed, how tested, inputs/outputs, and next teammate dependency. Never commit secrets, private source photos or model weights.

BUILD GUIDE / 11

Today’s build schedule

All times are Atlanta / EDT. This is a deadline-based schedule, replacing the old 26-hour plan. If work starts late, cut optional scope rather than the submission buffer.

TIME	PARALLEL WORK	SHARED EXIT GATE
12:30-13:00	A scaffold; B provider check; C render test; D repo/Auth0/DB.	Names, contracts, photo and integration host agreed.
13:00-14:30	A fixtures/screens; B real scene output; C one edit; D authenticated upload.	14:30: first joint merge; image go/no-go decision.
14:30-17:00	A polling/slider; B review/draft; C output adapter; D jobs/publish.	17:00: live upload → scene → mask → preview.
17:00-20:00	A support/packet view; B Gemini; C regression; D unique support/PDF/hosting.	20:00: full loop twice with two accounts.
20:00-22:00	Polish, mobile check, provider failure handling; analytics only if core stable.	22:00: sponsor choice and feature freeze.
22:00-00:00	Record video, README, fallback example and submission draft.	Demo assets complete; no new dependencies.
00:00-05:30	Arrange meaningful rest coverage as a team.	Keep a working build; fix only urgent blockers.
05:30-07:00	Three rehearsals and final link checks.	Fresh-browser demo passes.
07:00-07:30	D submits; teammate verifies.	Devpost + required event form completed.
07:30-08:30	Buffer for link or submission issues.	Confirm roster, video, repo and demo access.
09:00	Official deadline.	All materials already submitted.

Checkpoint format: 10 minutes together

A demonstrates the current UI. D runs the live endpoints. B/C show the actual provider outputs. Report exact mismatches: "job succeeded but concept is null" is actionable; "backend broken" is not. Agree one owner and a time for each blocker.

BUILD GUIDE / 12

Acceptance checks and demo

Test the boundaries together. A convincing demonstration requires real outputs and reliable state, not just completed individual screens.

• Fresh browser: upload → suggestion → mask → genuine edit → review → publish → support → packet.

• A's sample responses and D's live responses match the agreed schema.

• User 2 cannot access User 1's draft, job or private image.

• User 2 supports once; repeat and simultaneous duplicate requests do not increase the count. A third account contributes one more.

• Source and concept retain separate IDs/hashes. No source replacement during regeneration.

• Old preview output cannot overwrite newer draft content; old publish revision returns a conflict.

• Render failure shows retry/error, not a silent prepared image.

• Packet includes both labeled images, exact request, timestamped count and recipient uncertainty.

• Refresh/restart preserves proposal data and handles abandoned jobs visibly.

Three-minute demo

TIME	SHOW
0:00-0:20	The actual place and one specific problem.
0:20-0:50	Upload photo and select shade.
0:50-1:25	Real generation and before/after comparison; disclose if using a prepared backup.
1:25-1:55	Review exact wording and flags, then publish.
1:55-2:25	Second account supports; repeated click leaves count unchanged.
2:25-3:00	Download the packet; explain actual sponsor use and prototype limit.

Cut order if behind: map → comments → advanced charts → second concept → unproven third sponsor. Preserve source/concept comparison, actual edit, protected publish/support and an honest export.

BUILD GUIDE / 13

Start now: everyone’s next action

The companion README and Cursor prompts contain the detailed implementation instructions. Do this first before anyone starts a separate app.

OWNER	NEXT 30-60 MINUTES
A	Open the shared clone and frontend branch. Scaffold React/Vite once. Read screen handoffs; build Capture and Preview against fixtures. Create the shared API client before scattering fetch calls.
B	Open agents branch. Test actual local model availability. Return one schema-valid scene option and preserve list. Share function signature and errors with D.
C	Open image branch. Get the consented source and matching mask. Run one actual edit; save workflow, model identifier and elapsed time. Report feasibility, not just installation progress.
D	Create/invite to repo; merge contract kit. Set up Django and PostgreSQL, Auth0 audience and verified token path. Provide create/read responses matching fixtures. Decide integration host with C.

End-of-task handoff template

I changed: files and behavior.
You call: endpoint or function.
You send: exact fields/example.
You receive: schema/fixture.
It needs: environment variables and services, without secrets.
I verified: real test and result.
Still blocked: one concrete dependency and owner.

Final submission

Public repository and README; working demo or clear local instructions; public 2-3 minute video; all four accepted participants; chosen main track and at most three side challenges; prior-code/asset disclosures; Devpost plus required organizer form. Confirm latest Discord instructions. [1]

Sources and status

[1] User-supplied Hacklanta 2026 Hacker Guide: categories, entry cap, deadline, submission requirements.

[2] Auth0 React quickstart: https://auth0.com/docs/quickstart/spa/react

[3] Vite setup: https://vite.dev/guide/

[4] Django REST Framework: https://www.django-rest-framework.org/api-guide/views/

Inputs: PlacePatch_Hacklanta_2026_Team_Plan (1).pdf and IMG_0776.jpg. Technical defaults and contracts above are the proposed v2 agreement; names, models, hardware and accounts still need filling at kickoff.